

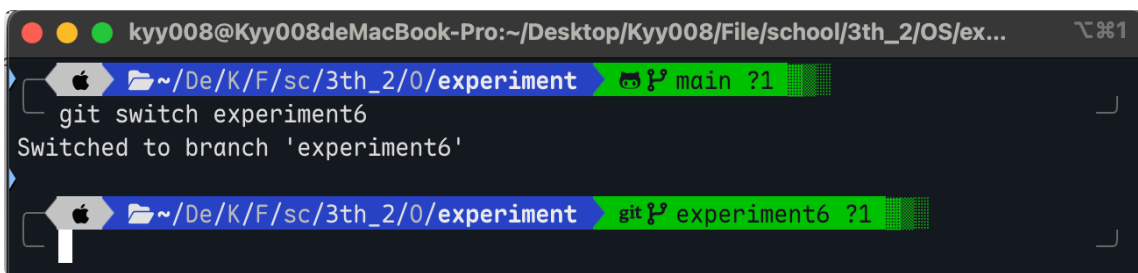
2026 春《操作系统》课程实验报告

实验 6

23301037 客昱阳

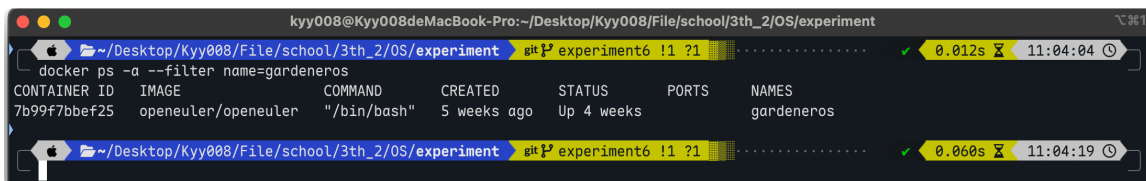
一.准备工作

- 将分支切换到实验四所需的 `experiment6` 分支



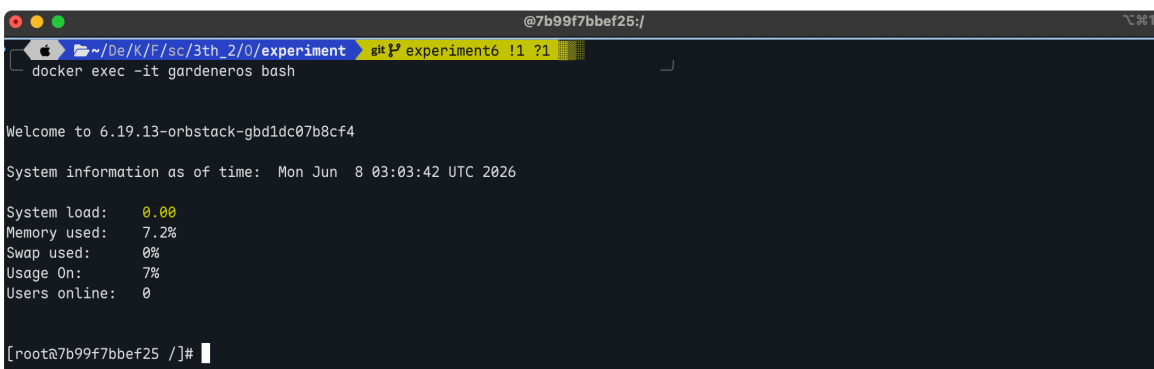
```
kyy008@Kyy008deMacBook-Pro:~/Desktop/Kyy008/File/school/3th_2/OS/ex...
~/De/K/F/sc/3th_2/0/experiment git main ?1
git switch experiment6
Switched to branch 'experiment6'
~/De/K/F/sc/3th_2/0/experiment git experiment6 ?1
```

- 检查一下上次环境配置的 docker 容器已经启动



```
kyy008@Kyy008deMacBook-Pro:~/Desktop/Kyy008/File/school/3th_2/OS/experiment
~/Desktop/Kyy008/File/school/3th_2/OS/experiment git experiment6 !1 ?1
docker ps -a --filter name=gardeneros
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
7b99f7bbef25   openeuler/openeuler  "/bin/bash"             5 weeks ago   Up 4 weeks   -       gardeneros
~/Desktop/Kyy008/File/school/3th_2/OS/experiment git experiment6 !1 ?1
```

- 进入容器



```
@7b99f7bbef25:/
~/De/K/F/sc/3th_2/0/experiment git experiment6 !1 ?1
docker exec -it gardeneros bash

Welcome to 6.19.13-orbstack-gbd1dc07b8cf4

System information as of time: Mon Jun  8 03:03:42 UTC 2026

System load:   0.00
Memory used:   7.2%
Swap used:     0%
Usage On:      7%
Users online:  0

[root@7b99f7bbef25 /]#
```

二.实验步骤

由于实验较长，大部分步骤不给出具体代码，详情见 git 仓库。

1.在内核中支持动态内存分配

- 首先在 `os/Cargo.toml` 中增加伙伴分配器依赖。内核没有标准库支持，后续需要借助 `buddy_system_allocator` 实现动态内存分配。

Bash

```
1 cd /mnt
2 vim os/Cargo.toml
```

在 `[dependencies]` 下加入：

```
[dependencies]
buddy_system_allocator = "0.6"
lazy_static = { version = "1.4.0", features = ["spin_no_std"] }
riscv = { git = "https://github.com/rcore-os/riscv", features = ["inline-asm"] }
```

- 在 `main.rs` 中引入 `alloc` 库，并开启动态内存分配失败处理所需的特性。后续内核中的 `Box`、`Vec` 等动态分配结构都依赖 `alloc`

Bash

```
1 vim os/src/main.rs
```

将开头修改为：

```
#![no_std]
#![no_main]
#![feature(alloc_error_handler)]

extern crate alloc;

use core::arch::global_asm;
```

- 在 `config.rs` 中增加内核堆空间大小。后续全局堆分配器会使用这块空间进行动态内存分配。

Bash

```
1 vim os/src/config.rs
```

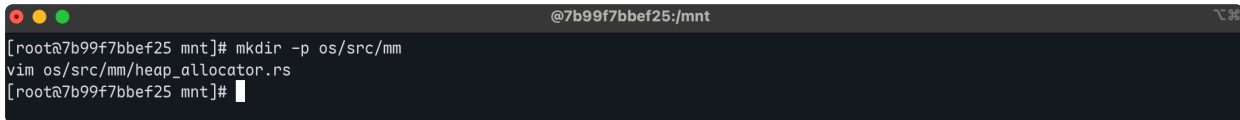
末尾加入：

```
pub const APP_SIZE_LIMIT: usize = 0x20000;
pub const CLOCK_FREQ: usize = 12500000;
pub const KERNEL_HEAP_SIZE: usize = 0x30_0000;
```

- 创建 `mm` 模块目录，并实现内核堆分配器。这里使用 `buddy_system_allocator` 提供的伙伴分配器作为全局动态内存分配器。

Bash

```
1 mkdir -p os/src/mm
2 vim os/src/mm/heap_allocator.rs
```

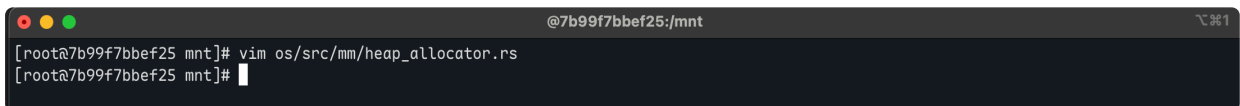


```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# mkdir -p os/src/mm
vim os/src/mm/heap_allocator.rs
[root@7b99f7bbef25 mnt]#
```

- 在 `heap_allocator.rs` 中加入动态内存分配测试函数。该函数通过 `Box` 和 `Vec` 测试内核堆是否可以正常申请和释放内存。

Bash

```
1 vim os/src/mm/heap_allocator.rs
```

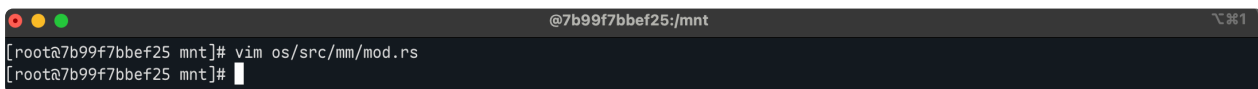


```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/heap_allocator.rs
[root@7b99f7bbef25 mnt]#
```

- 创建 `mm/mod.rs`，初始化内核堆分配器，并调用 `heap_test` 测试动态内存分配是否正常。

Bash

```
1 vim os/src/mm/mod.rs
```



```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/mod.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `main.rs`，引入 `mm` 模块，并在内核初始化时调用 `mm::init()` 初始化堆分配器。

Bash

```
1 vim os/src/main.rs
```



```
#![no_mangle]
pub fn rust_main() -> ! {
    clear_bss();
    mm::init();
    println!("[kernel] Hello, world!");
    loop {}
}
```

2.实现虚拟地址与物理地址的基本定义

- 创建 `address.rs`，定义物理地址、虚拟地址、物理页号和虚拟页号四种基本类型。后续页表和物理帧管理都会基于这些类型实现。

Bash

```
1 vim os/src/mm/address.rs
```

```
"os/src/mm/address.rs" 15L, 401B                                15,34      All
#[repr(C)]
#[derive(Copy, Clone, Ord, PartialOrd, Eq, PartialEq)]
pub struct PhysAddr(pub usize);

#[repr(C)]
#[derive(Copy, Clone, Ord, PartialOrd, Eq, PartialEq)]
pub struct VirtAddr(pub usize);

#[repr(C)]
#[derive(Copy, Clone, Ord, PartialOrd, Eq, PartialEq)]
pub struct PhysPageNum(pub usize);

#[repr(C)]
#[derive(Copy, Clone, Ord, PartialOrd, Eq, PartialEq)]
pub struct VirtPageNum(pub usize);
```

- 在 `address.rs` 中实现地址类型和 `usize` 之间的相互转换，方便后续在地址、页号和原始整数之间进行转换。

Bash

```
1 vim os/src/mm/address.rs
```

```
[root@7b99f7bbef25 mnt]# vim os/src/mm/address.rs
[root@7b99f7bbef25 mnt]#
```

- 在 `config.rs` 中增加页大小相关常量。后续地址和页号之间的转换会使用这些常量。

Bash

```
1 vim os/src/config.rs
```

```
pub const PAGE_SIZE: usize = 0x1000;
pub const PAGE_SIZE_BITS: usize = 0xc;
```

- 在 `address.rs` 中实现地址和页号之间的相互转换。`floor` 用于向下取整页号，`ceil` 用于向上取整页号，`page_offset` 用于取得页内偏移。

Bash

```
1 vim os/src/mm/address.rs
```

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/address.rs
[root@7b99f7bbef25 mnt]#
```

- 在 `address.rs` 中继续实现地址类型的调试输出、虚拟页号索引计算，以及页号范围迭代器。后续多级页表管理会用这些工具函数遍历虚拟页号范围。

Bash

```
1 vim os/src/mm/address.rs
```

3. 定义页表项数据结构

- 在 `main.rs` 中引入 `bitflags` 宏。后续页表项中的标志位会使用 `bitflags!` 定义。

Bash

```
1 vim os/src/main.rs
```

添加以下内容：

```
#![no_std]
#![no_main]
#![feature(alloc_error_handler)]

extern crate alloc;
#[macro_use]
extern crate bitflags;

use core::arch::global_asm;
mod mm;
```

- 在 `os/Cargo.toml` 中增加 `bitflags` 依赖。页表项中的 V、R、W、X 等标志位会通过该库定义。

Bash

```
1 vim os/Cargo.toml
```

添加以下内容：

```
[dependencies]
bitflags = "1.2.1"
buddy_system_allocator = "0.6"
lazy_static = { version = "1.4.0", features = ["spin_no_std"] }
riscv = { git = "https://github.com/rcore-os/riscv", features = ["inline-asm"] }
```

- 创建 `page_table.rs`，先定义页表项中的标志位。V 表示有效位，R/W/X 表示读、写、执行权限，U 表示用户态可访问。

Bash

```
1 vim os/src/mm/page_table.rs
```

```
use bitflags::*;

bitflags! {
    pub struct PTEFlags: u8 {
        const V = 1 << 0;
        const R = 1 << 1;
        const W = 1 << 2;
        const X = 1 << 3;
        const U = 1 << 4;
        const G = 1 << 5;
        const A = 1 << 6;
        const D = 1 << 7;
    }
}
```

- 继续在 `page_table.rs` 中定义 `PageTableEntry`。页表项中保存物理页号和标志位，后续页表映射会通过它记录虚拟页到物理页的映射关系。

Bash

```
1 vim os/src/mm/page_table.rs
```

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/page_table.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `mm/mod.rs`，把 `address.rs` 和 `page_table.rs` 接入 `mm` 模块，并导出后续物理帧管理和页表管理要用到的类型。

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/mod.rs
[root@7b99f7bbef25 mnt]#
```

4.实现物理帧的管理与分配

(1)设置物理内存的终止地址

- 在 `config.rs` 中增加物理内存结束地址。后续物理帧分配器会用这个地址确定可分配物理页帧的范围。

Bash

```
1 vim os/src/config.rs
```

添加：

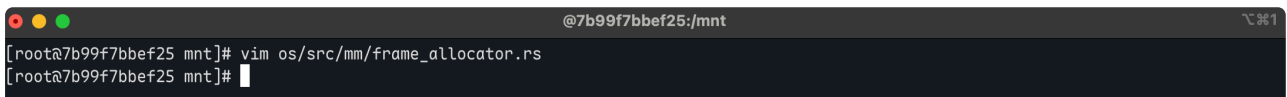
```
pub const PAGE_SIZE: usize = 0x1000;
pub const PAGE_SIZE_BITS: usize = 0xc;
pub const MEMORY_END: usize = 0x80800000;
```

(2)实现物理帧管理

- 创建 `frame_allocator.rs`，实现物理帧分配器。

Bash

```
1 vim os/src/mm/frame_allocator.rs
```



```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/frame_allocator.rs
[root@7b99f7bbef25 mnt]#
```

(3)增加 sync 模块

- 创建 sync 模块，用来提供 `UPSafeCell`。

Bash

```
1 mkdir -p os/src/sync
2 vim os/src/sync/up.rs
```



```
"os/src/sync/up.rs" 19L, 363B                                19,1      All
use core::cell::{RefCell, RefMut};

pub struct UPSafeCell<T> {
    inner: RefCell<T>,
}

unsafe impl<T> Sync for UPSafeCell<T> {}

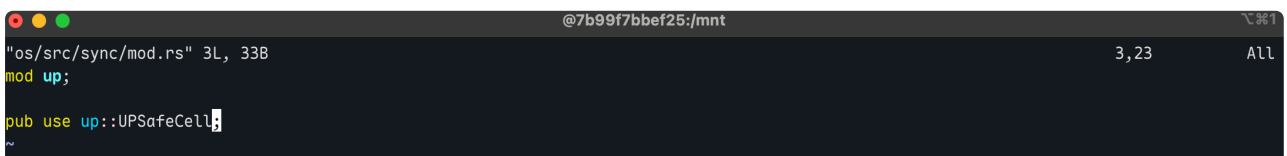
impl<T> UPSafeCell<T> {
    pub unsafe fn new(value: T) -> Self {
        Self {
            inner: RefCell::new(value),
        }
    }

    pub fn exclusive_access(&self) -> RefMut<'_, T> {
        self.inner.borrow_mut()
    }
}
```

- 创建 `sync/mod.rs`，导出 `UPSafeCell`

Bash

```
1 vim os/src/sync/mod.rs
```



```
@7b99f7bbef25:/mnt
"os/src/sync/mod.rs" 3L, 33B                                3,23      All
mod up;

pub use up::UPSafeCell;
```

- 修改 `main.rs`，引入 `sync` 模块

Bash

```
1 vim os/src/main.rs
```

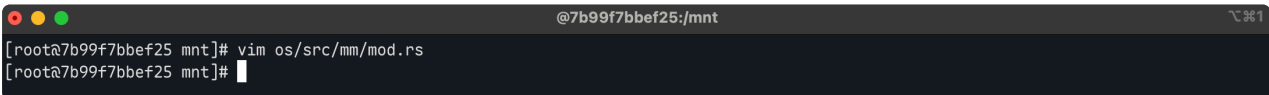
```
#[macro_use]
mod console;
mod lang_items;
mod sbi;
mod syscall;
mod trap;
mod loader;
mod config;
mod task;
mod timer;
mod sync;
```

(4)物理帧管理测试

- 修改 `mm/mod.rs`，接入物理帧分配器，并在初始化时运行测试。

Bash

```
1 vim os/src/mm/mod.rs
```



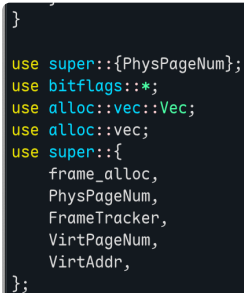
```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/mod.rs
[root@7b99f7bbef25 mnt]#
```

5.多级页表管理

- 修改 `page_table.rs`，引入后续页表管理需要使用的类型和 `Vec`。

Bash

```
1 vim os/src/mm/page_table.rs
```



```
}

use super::{PhysPageNum};
use bitflags::*;
use alloc::vec::Vec;
use alloc::vec;
use super::{
    frame_alloc,
    PhysPageNum,
    FrameTracker,
    VirtPageNum,
    VirtAddr,
};
```

`PageTable` 结构体，并实现创建新页表的方法。


```
pub struct PageTable {
    root_ppn: PhysPageNum,
    frames: Vec<FrameTracker>,
}

impl PageTable {
    pub fn new() -> Self {
        let frame = frame_alloc().unwrap();
        PageTable {
            root_ppn: frame.ppn,
            frames: vec![frame],
        }
    }
}
```

85,1

Bot

实现建立和拆除虚实地址之间映射关系的方法。

```
[root@7b99f7bbef25 mnt]# vim os/src/mm/page_table.rs
[root@7b99f7bbef25 mnt]#
```

实现手动查询页表的方法。

```
[root@7b99f7bbef25 mnt]# vim os/src/mm/page_table.rs
[root@7b99f7bbef25 mnt]#
```

6.内核与应用的地址空间

(1)实现地址空间抽象

- 创建 memory_set.rs, 定义地址空间相关的基本数据结构。

Bash

```
1 vim os/src/mm/memory_set.rs
```

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/memory_set.rs
[root@7b99f7bbef25 mnt]#
```

- 在 memory_set.rs 中实现 MemorySet 的基本方法。

Bash

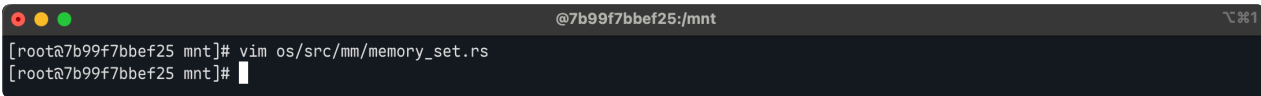
```
1 vim os/src/mm/memory_set.rs
```

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/memory_set.rs
[root@7b99f7bbef25 mnt]#
```

- 在 memory_set.rs 中实现跳板页映射、地址空间激活和页表查询。

Bash

```
1 vim os/src/mm/memory_set.rs
```

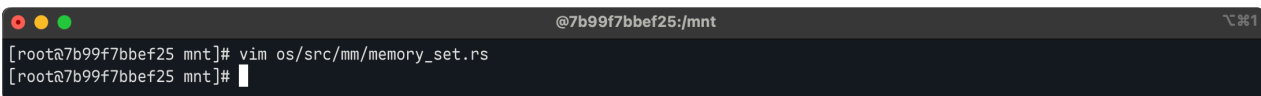


```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/memory_set.rs
[root@7b99f7bbef25 mnt]#
```

- 在 `memory_set.rs` 中实现 `MapArea` 的相关方法。

Bash

```
1 vim os/src/mm/memory_set.rs
```



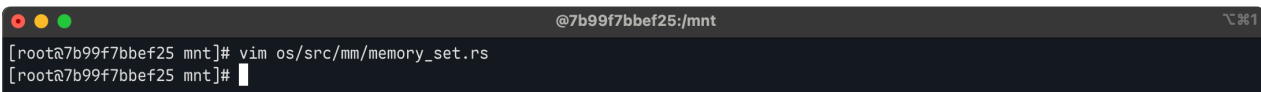
```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/memory_set.rs
[root@7b99f7bbef25 mnt]#
```

(2)实现内核地址空间

- 在 `memory_set.rs` 中实现创建内核地址空间的方法 `new_kernel`。

Bash

```
1 vim os/src/mm/memory_set.rs
```



```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/memory_set.rs
[root@7b99f7bbef25 mnt]#
```

(3)实现应用地址空间

- 修改用户程序链接脚本，将应用程序起始地址改为 `0x0`。

Bash

```
1 vim user/src/linker.lds
```



```
OUTPUT_ARCH(riscv)
ENTRY(_start)
BASE_ADDRESS = 0x0

SECTIONS
{
    . = BASE_ADDRESS;
    .text : {
        *(.text.entry)
        *(.text .text.*)
    }
}
```

- 修改 `loader.rs`，精简为直接读取应用 ELF 数据。

Bash

```
1 vim os/src/loader.rs
```

```
[root@7b99f7bbef25 mnt]# vim os/src/loader.rs
[root@7b99f7bbef25 mnt]#
```

- 在 `os/Cargo.toml` 中增加 `xmas-elf` 依赖。

Bash

```
1 vim os/Cargo.toml
```

```
[dependencies]
xmas-elf = "0.7.0"
bitflags = "1.2.1"
buddy_system_allocator = "0.6"
lazy_static = { version = "1.4.0", features = ["spin_no_std"] }
riscv = { git = "https://github.com/rcore-os/riscv", features = ["inline-asm"] }
```

- 在 `memory_set.rs` 中实现解析 `ELF`，创建应用地址空间的方法。

Bash

```
1 vim os/src/mm/memory_set.rs
```

```
[root@7b99f7bbef25 mnt]# vim os/src/mm/memory_set.rs
[root@7b99f7bbef25 mnt]#
```

7.实现基于地址空间的分时多任务

(1)建立基于分页模式的虚拟地址空间

- 在 `memory_set.rs` 中创建内核地址空间 `KERNEL_SPACE`。

Python

```
1 vim os/src/mm/memory_set.rs
```

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/memory_set.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `mm/mod.rs`，接入 `memory_set` 子模块，并在初始化时激活内核地址空间。

Python

```
1 vim os/src/mm/mod.rs
```

```
[root@7b99f7bbef25 mnt]# vim os/src/mm/mod.rs
[root@7b99f7bbef25 mnt]#
```

- 在 `memory_set.rs` 中实现 `remap_test`，检查内核地址空间映射权限。

Python

```
1 vim os/src/mm/memory_set.rs
```

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/memory_set.rs
[root@7b99f7bbef25 mnt]#
```

(2)实现跳板机制

- 修改 `trap/context.rs`，扩展 `TrapContext`，保存内核页表、内核栈和 trap 处理函数地址。

Python

```
1 vim os/src/trap/context.rs
```

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/trap/context.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `trap.S`，实现跳板机制下的 trap 进入和返回。

Python

```
1 vim os/src/trap/trap.S
```

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/trap/trap.S
[root@7b99f7bbef25 mnt]#
```

- 修改 `linker.ld`，将跳板代码放入单独的 `.text.trampoline` 段并按页对齐

Python

```
1 vim os/src/linker.ld
```

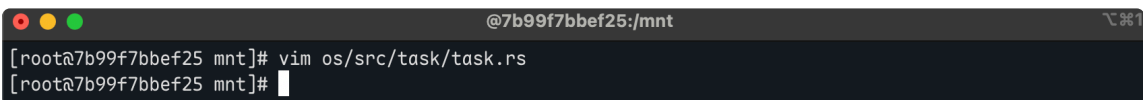
```
stext = .;
.text : {
    *(.text.entry)
    . = ALIGN(4K);
    strampoline = .;
    *(.text.trampoline);
    . = ALIGN(4K);
    *(.text .text.*)
}
```

(3)加载和执行应用程序

修改 `task/task.rs`，更新任务控制块，使每个任务拥有独立地址空间和 Trap 上下文页。

Python

```
1 vim os/src/task/task.rs
```

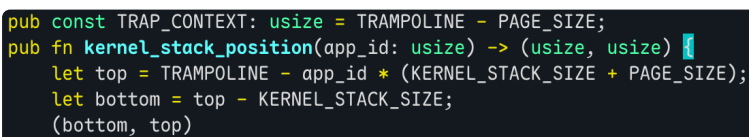


```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/task.rs
[root@7b99f7bbef25 mnt]#
```

- 在 `config.rs` 中增加根据应用编号计算内核栈位置的函数。

Python

```
1 vim os/src/config.rs
```

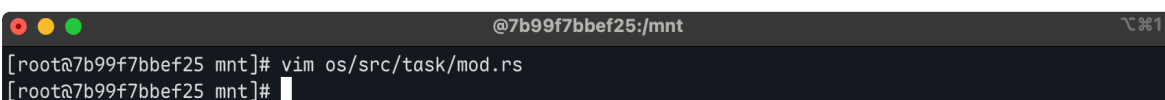


```
pub const TRAP_CONTEXT: usize = TRAMPOLINE - PAGE_SIZE;
pub fn kernel_stack_position(app_id: usize) -> (usize, usize) {
    let top = TRAMPOLINE - app_id * (KERNEL_STACK_SIZE + PAGE_SIZE);
    let bottom = top - KERNEL_STACK_SIZE;
    (bottom, top)
}
```

- 修改 `task/mod.rs`，使用 `Vec<TaskControlBlock>` 管理所有应用任务。

Python

```
1 vim os/src/task/mod.rs
```

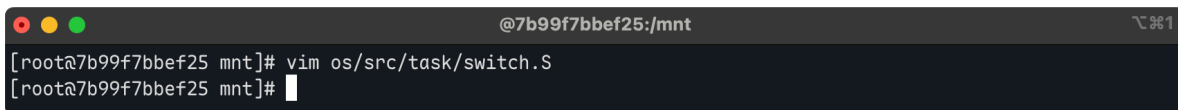


```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/mod.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `task/switch.S`，保存和恢复任务的内核栈指针。

Python

```
1 vim os/src/task/switch.S
```

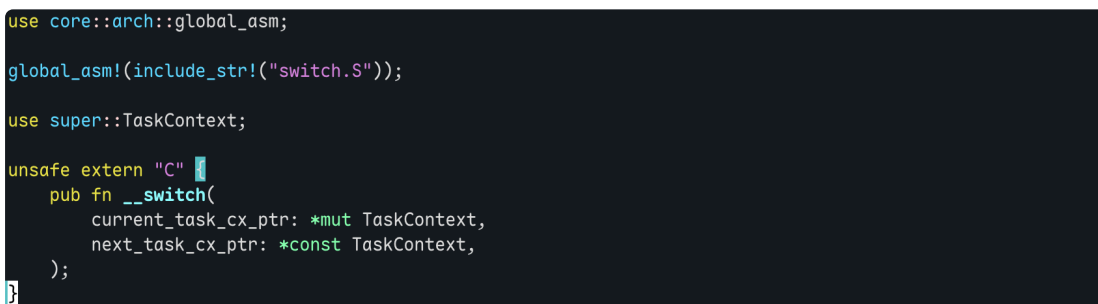


```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/switch.S
[root@7b99f7bbef25 mnt]#
```

- 修改 `task/switch.rs`，更新 `__switch` 的函数签名。

Python

```
1 vim os/src/task/switch.rs
```



```
use core::arch::global_asm;

global_asm!(include_str!("switch.S"));

use super::TaskContext;

unsafe extern "C" {
    pub fn __switch(
        current_task_cx_ptr: *mut TaskContext,
        next_task_cx_ptr: *const TaskContext,
    );
}
```

(4)改进 Trap 的处理

- 修改 `trap/mod.rs`，引入地址空间切换后 trap 处理需要用到的模块和常量。

Python

```
1 vim os/src/trap/mod.rs
```



```
use crate::task::{
    current_user_token,
    current_trap_cx,
};
use crate::config::{
    TRAP_CONTEXT,
    TRAMPOLINE,
};
use core::arch::global_asm;
use riscv::register::{
```

- 修改 `trap/mod.rs`，设置内核态和用户态 trap 入口。
- 修改 `trap/mod.rs`，更新 `trap_handler`，处理完 trap 后调用 `trap_return()` 返回用户态。
- 修改 `trap/mod.rs`，实现 `trap_return()` 和 `trap_from_kernel()`

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/trap/mod.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `task/context.rs`，让任务第一次运行时进入 `trap_return()`。

Python

```
1 vim os/src/task/context.rs
```

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/context.rs
[root@7b99f7bbef25 mnt]#
```

(5)改进 sys_write 的实现

- 在 `page_table.rs` 中实现 `translated_byte_buffer`，用于把用户地址空间中的缓冲区转换为内核可访问的切片。

Python

```
1 vim os/src/mm/page_table.rs
```

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/mm/page_table.rs
[root@7b99f7bbef25 mnt]#
```

- 修改 `syscall/fs.rs`，让 `sys_write` 通过页表访问用户缓冲区。

Python

```
1 vim os/src/syscall/fs.rs
```

```
"os/src/syscall/fs.rs" 18L, 448B      1,1      All
use crate::mm::translated_byte_buffer;
use crate::task::current_user_token;

const FD_STDOUT: usize = 1;

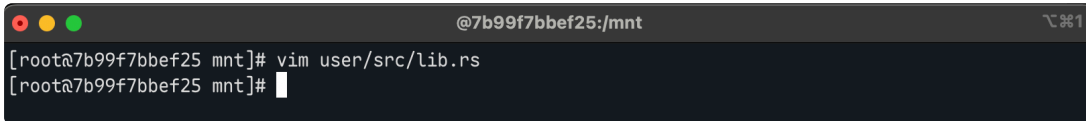
pub fn sys_write(fd: usize, buf: *const u8, len: usize) -> isize {
    match fd {
        FD_STDOUT => {
            let buffers = translated_byte_buffer(current_user_token(), buf, len);
            for buffer in buffers {
                print!("{}", core::str::from_utf8(buffer).unwrap());
            }
            len as isize
        }
        _ => {
            panic!("Unsupported fd in sys_write!");
        }
    }
}
```

8.修改应用程序

- 修改 `user/src/lib.rs`，删除用户程序中的 `clear_bss()`，并删除 `_start()` 中对 `clear_bss()` 的调用。

Bash

```
1 vim user/src/lib.rs
```

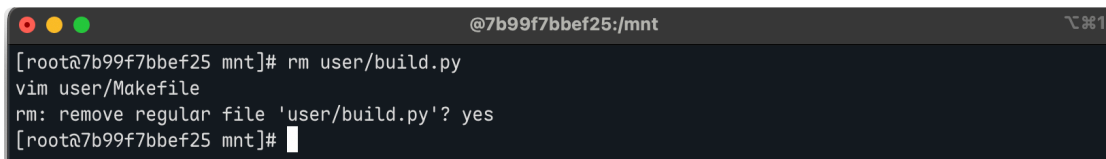


```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim user/src/lib.rs
[root@7b99f7bbef25 mnt]#
```

- 删除 `user/build.py`，并修改 `user/Makefile` 中的 `elf` 规则。

Bash

```
1 rm user/build.py
2 vim user/Makefile
```



```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# rm user/build.py
vim user/Makefile
rm: remove regular file 'user/build.py'? yes
[root@7b99f7bbef25 mnt]#
```

9.修改 main.rs

- 修改 `os/src/main.rs`，调整初始化顺序，并加入 `remap_test()`。

Bash

```
1 vim os/src/main.rs
```



```
#[unsafe(no_mangle)]
pub fn rust_main() -> ! {
    clear_bss();
    println!("[kernel] Hello, world!");
    mm::init();
    println!("[kernel] back to world!");
    mm::remap_test();
    trap::init();
    trap::enable_timer_interrupt();
    timer::set_next_trigger();
    task::run_first_task();
    panic!("Unreachable in rust_main!");
}
```

- 修改 `os/build.rs`，让内核直接打包用户程序 ELF 文件。

Bash

```
1 vim os/build.rs
```

```
[root@7b99f7bbef25 mnt]# vim os/build.rs
[root@7b99f7bbef25 mnt]#
```

10.编译运行程序

- 编译用户程序：

Bash

```
1 cd user
2 make build
```

```
[root@7b99f7bbef25 mnt]# cd user
make build
   Compiling user_lib v0.1.0 (/mnt/user)
   Finished `release` profile [optimized] target(s) in 0.33s
rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/00power_3 --strip-all -O binary target/riscv64gc-unknown-none-elf/release/00power_3.bin; rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/01power_5 --strip-all -O binary target/riscv64gc-unknown-none-elf/release/01power_5.bin; rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/02power_7 --strip-all -O binary target/riscv64gc-unknown-none-elf/release/02power_7.bin; rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/03sleep --strip-all -O binary target/riscv64gc-unknown-none-elf/release/03sleep.bin;
[root@7b99f7bbef25 user]#
```

- 编译内核：

Bash

```
1 cd ../os
2 make build
```

```
@7b99f7bbef25:/mnt/os
warning: constant `APP_BASE_ADDRESS` is never used
--> src/config.rs:4:11
4 | pub const APP_BASE_ADDRESS: usize = 0x80400000;
  |           ^^^^^^^^^^^^^^^^^
warning: constant `APP_SIZE_LIMIT` is never used
--> src/config.rs:5:11
5 | pub const APP_SIZE_LIMIT: usize = 0x20000;
  |           ^^^^^^^^^^^^^
warning: variant `UnInit` is never constructed
--> src/task/task.rs:14:5
13 | pub enum TaskStatus {
14 |     UnInit,
   |     ^^^^^
   = note: `TaskStatus` has a derived impl for the trait `Clone`, but this is intentionally ignored during dead code analysis
warning: `os` (bin "os") generated 4 warnings
   Finished `release` profile [optimized] target(s) in 0.02s
[root@7b99f7bbef25 os]#
```

- 运行程序

Bash

```
1 make run
```

```
@7b99f7bbef25:/mnt/os
power_7 [30000/200000]
power_7 [40000/200000]
power_7 [50000/200000]
power_7 [60000/200000]
power_7 [70000/200000]
power_7 [80000/200000]
power_7 [90000/200000]
power_7 [100000/200000]
power_7 [110000/200000]
power_7 [120000/200000]
power_7 [130000/200000]
power_7 [140000/200000]
power_7 [150000/200000]
power_7 [160000/200000]
power_7 [170000/200000]
power_7 [180000/200000]
power_7 [190000/200000]
power_7 [200000/200000]
7^200000 = 277895943
Test power_7 OK!
[kernel] Application exited with code 0
Test sleep OK!
[kernel] Application exited with code 0
Panic at src/task/mod.rs:124 All applications completed!
[root@7b99f7bbef25 os]#
```

表明所有应用程序已经执行完成，至此，内存管理部分完成。

三.问题回答

1.分析虚拟地址和物理地址的设计与实现

本实验将地址分成四种类型：

Rust

```
1 pub struct PhysAddr(pub usize);
2 pub struct VirtAddr(pub usize);
3 pub struct PhysPageNum(pub usize);
4 pub struct VirtPageNum(pub usize);
```

其中，PhysAddr 表示物理地址，VirtAddr 表示虚拟地址，PhysPageNum 和 VirtPageNum 分别表示物理页号和虚拟页号。虽然它们底层都是 `usize`，但用不同类型封装后，可以避免在页表实现中混淆不同地址。

地址和页号之间通过页大小进行转换。本实验中页大小为 4KB：

Rust

```
1 pub const PAGE_SIZE: usize = 0x1000;
2 pub const PAGE_SIZE_BITS: usize = 0xc;
```

虚拟地址可以通过 `floor()`、`ceil()` 得到对应虚拟页号，也可以通过 `page_offset()` 得到页内偏移。

Rust

```
1 pub fn floor(&self) -> VirtPageNum {
2     VirtPageNum(self.0 / PAGE_SIZE)
3 }
```

在 SV39 分页机制中，一个虚拟页号会被拆成三级页表索引。代码中通过 `indexes()` 得到三个 9 位索引，用于多级页表查找。

Rust

```
1 idx[i] = vpn & 511;
2 vpn >>= 9;
```

物理页号还可以转换成页表项数组或字节数组，方便内核直接操作页表页和物理页内容。

因此，本实验通过封装地址类型、实现地址页号转换和三级索引拆分，为后续页表映射和地址空间管理提供了基础。

2.分析物理帧是如何管理与分配的；

物理帧管理主要由 `frame_allocator.rs` 实现。内核把 `ekernel` 到 `MEMORY_END` 之间的物理内存作为可分配区域。

Rust

```
1 .init(
2     PhysAddr::from(ekernel as *const () as usize).ceil(),
3     PhysAddr::from(MEMORY_END).floor(),
4 );
```

每个物理页帧用 `FrameTracker` 表示。创建 `FrameTracker` 时，会先把对应物理页清零，防止旧数据影响新分配的页面。

Rust

```
1 pub struct FrameTracker {  
2     pub ppn: PhysPageNum,  
3 }
```

物理帧分配器使用 StackFrameAllocator 实现。它用 current 表示当前还未分配的起始页号，用 end 表示可分配范围的结束页号，用 recycled 保存已经回收的页帧。

Rust

```
1 pub struct StackFrameAllocator {  
2     current: usize,  
3     end: usize,  
4     recycled: Vec<usize>,  
5 }
```

分配物理帧时，优先从 recycled 中取出回收页帧；如果没有可回收页帧，就从 current 开始分配新的页帧。

Rust

```
1 if let Some(ppn) = self.recycled.pop() {  
2     Some(ppn.into())  
3 } else {  
4     self.current += 1;  
5     Some((self.current - 1).into())  
6 }
```

释放物理帧时，通过 Drop 自动调用 frame_dealloc()，将页帧号放回 recycled 中。这样可以避免手动释放遗漏。

Rust

```
1 impl Drop for FrameTracker {  
2     fn drop(&mut self) {  
3         frame_dealloc(self.ppn);  
4     }  
5 }
```

因此，物理帧分配采用栈式管理方式，结合 FrameTracker 的自动回收机制，实现了物理页帧的分配、清零和释放。

3.分析内核的地址空间以及应用程序的地址空间是如何实现的；

本实验用 MemorySet 表示一个地址空间，其中包含一个页表 PageTable 和多个逻辑段 MapArea。

Rust

```
1 pub struct MemorySet {  
2     page_table: PageTable,  
3     areas: Vec<MapArea>,  
4 }
```

内核地址空间通过 MemorySet::new_kernel() 创建。它会映射跳板页、内核代码段、只读数据段、数据段、bss 段以及剩余物理内存。

Rust

```
1 memory_set.map_trampoline();
```

不同段设置不同权限，例如 .text 段只读可执行，.rodata 段只读，.data 和 .bss 段可读写。

Rust

```
1 MapPermission::R | MapPermission::X  
2 MapPermission::R | MapPermission::W
```

内核使用恒等映射，也就是虚拟页号和物理页号相同。

Rust

```
1 MapType::Identical
```

应用程序地址空间通过 MemorySet::from_elf() 创建。内核解析 ELF 文件，根据程序头把用户程序的代码段、数据段等加载到独立的用户地址空间中。

Rust

```
1 let elf = xmas_elf::ElfFile::new(elf_data).unwrap();
```

应用地址空间使用 Framed 映射，为每个虚拟页分配新的物理页帧，并把 ELF 中的数据复制进去。

Rust

```
1 MapType::Framed
```

此外，每个应用还会映射用户栈、TrapContext 页和跳板页。用户程序段会带有 U 权限，表示用户态可以访问。

Rust

```
1 MapPermission::R | MapPermission::W | MapPermission::U
```

因此，内核地址空间主要采用恒等映射，保证内核能访问物理内存；应用地址空间则根据 ELF 动态创建，并通过页表实现进程之间的地址隔离。

4.分析基于地址空间的分时多任务是如何实现的；

本实验中，每个任务都有独立的地址空间。任务控制块 TaskControlBlock 中保存了任务状态、任务上下文、地址空间和 TrapContext 所在物理页号。

Rust

```
1 pub struct TaskControlBlock {
2     pub task_status: TaskStatus,
3     pub task_cx: TaskContext,
4     pub memory_set: MemorySet,
5     pub trap_cx_ppn: PhysPageNum,
6 }
```

创建任务时，内核会通过 MemorySet::from_elf() 为应用程序建立独立用户地址空间，并得到入口地址和用户栈地址。

Rust

```
1 let (memory_set, user_sp, entry_point) = MemorySet::from_elf(elf_data);
```

任务切换仍然依赖 `__switch`，它保存当前任务的内核上下文，并恢复下一个任务的内核上下文。

Rust

```
1 __switch(  
2     current_task_cx_ptr,  
3     next_task_cx_ptr,  
4 );
```

和之前不同的是，进入用户态前需要切换到对应应用的地址空间。`trap_return()` 会取得当前任务的用户页表 token，并跳转到 trampoline 中的 `__restore`。

Rust

```
1 let user_satp = current_user_token();
```

在 `__restore` 中，内核将用户页表写入 `satp`，完成地址空间切换，然后恢复用户态寄存器并执行 `sret` 返回用户程序。

Rust

```
1 csrw satp, a1  
2 sfence.vma  
3 sret
```

发生系统调用或时钟中断时，trampoline 中的 `__alltraps` 会切换回内核页表，并进入 `trap_handler()`。如果是时钟中断，内核会调用 `suspend_current_and_run_next()` 切换到下一个任务。

Rust

```
1 Trap::Interrupt(Interrupt::SupervisorTimer) => {
2     set_next_trigger();
3     suspend_current_and_run_next();
4 }
```

因此，基于地址空间的分时多任务是在原有任务切换的基础上，为每个任务增加独立页表，并在 trap 返回和进入时完成用户地址空间与内核地址空间的切换。

5.编写新的应用程序并测试验证结果

- 程序一

Bash

```
1 vim user/src/bin/04addr_a.rs
```

Rust

```
1 #![no_std]
2 #![no_main]
3
4 #[macro_use]
5 extern crate user_lib;
6
7 static mut DATA: usize = 100;
8
9 #[unsafe(no_mangle)]
10 fn main() -> i32 {
11     unsafe {
12         DATA += 1;
13         println!(
14             "addr_a: DATA addr = {:#x}, value = {}",
15             &DATA as *const usize as usize,
16             DATA
17         );
18     }
19     println!("Test addr_a OK!");
20     0
21 }
```


- 程序二

Rust

```
1 #![no_std]
2 #![no_main]
3
4 #[macro_use]
5 extern crate user_lib;
6
7 static mut DATA: usize = 200;
8
9 #[unsafe(no_mangle)]
10 fn main() -> i32 {
11     unsafe {
12         DATA += 2;
13         println!(
14             "addr_b: DATA addr = {:#x}, value = {}",
15             &DATA as *const usize as usize,
16             DATA
17         );
18     }
19     println!("Test addr_b OK!");
20     0
21 }
```

- 重新编译和运行后得出结果

```
power_7 [120000/200000]
power_7 [130000/200000]
power_7 [140000/200000]
power_7 [150000/200000]
power_7 [160000/200000]
power_7 [170000/200000]
power_7 [180000/200000]
power_7 [190000/200000]
power_7 [200000/addr_a: DATA addr = 0x2000, value = 101
Test addr_a OK!
[kernel] Application exited with code 0
addr_b: DATA addr = 0x2000, value = 202
Test addr_b OK!
[kernel] Application exited with code 0
200000]
7^200000 = 277895943
Test power_7 OK!
[kernel] Application exited with code 0
Test sleep OK!
[kernel] Application exited with code 0
Panic! at src/task/mod.rs:124 All applications completed!
[root@7b92f7b9ef25_0s]#
```

通过新增 addr_a 和 addr_b 两个应用程序，分别修改各自的全局变量 DATA。运行结果显示两个程序中 DATA 的虚拟地址均为 0x2000，但变量值分别为 101 和 202，互不影响。因此可以验证每个应用拥有独立地址空间，虚拟地址相同并不代表物理地址相同。

四.Git 提交截图

Commits

experiment6

All users

All time

Commits on Jun 9, 2026

feat: 新增两个回答问题的测试程序

Kyy008 committed 1 minute ago

dae59b1

<>

feat: complete experient 5

Kyy008 committed 1 hour ago

ed53db8

<>

五.其他说明

无